UNIDAD 2 · BACK-END CON BASES DE DATOS · SEMANA 8

# Seguridad con Spring Security

*Configuración de roles y permisos*

Curso: Desarrollo Web Integrado (100000ST61)

Docente: Mag. Ing. Marco Antonio Flores Torres · Universidad Tecnológica del Perú

Autenticación | Autorización | Roles | Permisos

RUTA DE LA SESIÓN

## Metodología de la clase de hoy

Seguimos la ruta pedagógica de 5 fases de UTP para construir el aprendizaje de forma progresiva.

| Inicio | Utilidad | Transformación | Práctica | Cierre |
|---|---|---|---|---|
| 15 min | 15 min | 55 min | 35 min | 15 min |
| Rompe hielo, repaso y saberes previos | Logro de la sesión y su importancia | Spring Security, roles y permisos | Ejercicios progresivos con solución | Conclusiones, quiz y casos prácticos |

INICIO

# Rompe hielo: "¿Quién puede entrar?"

### Actividad en equipos (5 min)

- Formen equipos de 3 estudiantes. Imaginen un centro empresarial con 3 puertas: Recepción, Oficinas y Bóveda de sistemas.
- Cada equipo recibe 3 "credenciales": Visitante, Empleado y Administrador de TI.
- En 3 minutos, decidan qué credencial debería abrir cada puerta y por qué.
- Un vocero por equipo comparte su decisión en voz alta.

### ¿Por qué importa?

Cada decisión que tomaron ahora mismo -sin saberlo- es exactamente lo que hace Spring Security en una aplicación web: decidir quién entra, a qué recurso, y con qué nivel de confianza.



INICIO

# Repaso rápido: Sesión 7 (JPQL y Transacciones)

Tres preguntas sencillas para activar lo aprendido la clase anterior. Respondan de forma individual, en voz alta.

1. ¿Qué es JPQL y en qué se diferencia de SQL?
2. ¿Qué anotación usamos para indicar que un método del Service debe ejecutarse como una transacción?
3. Si dentro de un método @Transactional ocurre una excepción, ¿qué pasa con los cambios ya realizados en la base de datos?

INICIO

# ¿Por qué la seguridad no es opcional?

Cada API REST que construyen desde la Sesión 1 hasta hoy ha estado completamente abierta: cualquier persona con la URL puede leer, crear, modificar o borrar datos. En un proyecto real, eso es una vulnerabilidad crítica.



**74%**

de las brechas de seguridad involucran un factor humano o de credenciales mal gestionadas (Verizon DBIR).



**1ra**

línea de defensa de cualquier back-end: controlar quién se autentica y qué puede autorizar hacer.



**100%**

de los proyectos profesionales con Spring Boot que revisan roles/permisos en la entrevista técnica.



INICIO

# Saberes previos: la seguridad que ya usan a diario

Antes de programar nada, reconozcamos que ya interactúan con sistemas de roles y permisos todos los días.

**Yape / Plin**

Tú solo ves y mueves TU dinero. No puedes ver el saldo de otro usuario: eso es autorización a nivel de datos.

**Marketplace (Mercado Libre, Falabella)**

Un cliente compra; un vendedor publica productos; un administrador banea cuentas. Tres roles, tres capacidades distintas.

**Banca por internet**

Iniciar sesión (autenticación) no es lo mismo que poder aprobar una transferencia de alto monto (autorización adicional).

*Pregunta detonante: si Yape no verificara quién eres, ¿qué pasaría con tu dinero?*

UTILIDAD

# Repaso retador: Sesión 7

Ahora pensemos un poco más. Discutan en pareja antes de responder (2 min por pregunta).

1. ¿Por qué una consulta JPQL con JOIN puede fallar en tiempo de ejecución aunque su sintaxis sea correcta?

2. En un método de servicio que hace dos operaciones de escritura (descontar stock + registrar venta), ¿por qué NO basta con que cada operación tenga su propio try/catch si no usamos @Transactional?

3. ¿Qué diferencia práctica existe entre una consulta JPQL definida con @Query y un método derivado como findByNombreContaining()?



UTILIDAD

# Logro de la sesión

Al finalizar la sesión, el estudiante configura la seguridad de una aplicación Spring Boot mediante Spring Security, definiendo usuarios, roles y reglas de autorización, de modo que pueda proteger endpoints y recursos según el perfil del usuario autenticado.

**Esto se traduce en tres capacidades concretas al final de la clase:**

### Autenticar

Definir usuarios, contraseñas cifradas y validarlos al iniciar sesión.

### Autorizar

Asignar roles (ADMIN, VENDEDOR, CLIENTE) y permisos según el perfil.

### Proteger rutas

Restringir endpoints y acciones de acuerdo con el rol del usuario.

UTILIDAD

# Utilidad en el mundo real y tecnológico

### Sistemas de salud

Un enfermero ve el historial clínico; solo un médico puede registrar un diagnóstico. RBAC (control por roles) es exigido por ley en muchos países.

### Banca y fintech

Un cajero registra operaciones; solo un supervisor aprueba montos altos. Los roles delimitan responsabilidad y auditoría.

### E-commerce

El panel de administración de productos, precios y pedidos (como el de TechStore Perú SAC) debe estar oculto para clientes comunes.

### Plataformas SaaS

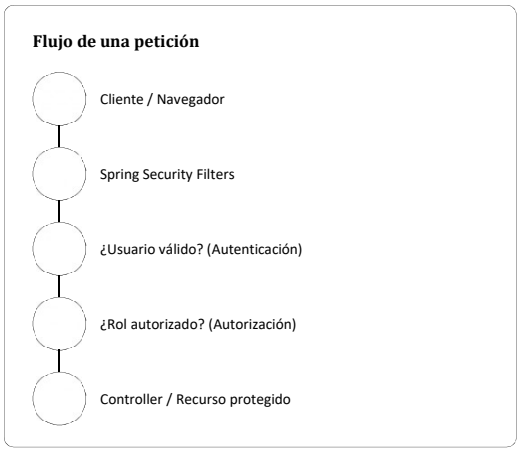
Cada empresa cliente (tenant) solo debe ver y modificar sus propios datos: autorización a nivel de negocio.



TRANSFORMACIÓN

# ¿Qué es Spring Security?

- Es el framework de seguridad del ecosistema Spring, integrado de forma nativa con Spring Boot.
- Se implementa internamente mediante una cadena de Servlet Filters que intercepta cada petición HTTP antes de que llegue al controlador.
- Resuelve dos problemas distintos pero complementarios: autenticación y autorización.
- Puede configurarse de forma declarativa (con @Configuration y beans de Java) o programática, según la necesidad del proyecto.



TRANSFORMACIÓN

# Autenticación vs. Autorización

Son los dos pilares de la seguridad, y suelen confundirse. No son lo mismo.

### Autenticación

*¿Quién eres?*

Verificar la identidad del usuario comparando sus credenciales (usuario + contraseña) con las almacenadas en el sistema.

**Ej: iniciar sesión en TechStore con tu usuario y contraseña.**

### Autorización

*¿Qué puedes hacer?*

Una vez identificado, determinar si el usuario tiene el rol o permiso necesario para acceder a un recurso o ejecutar una acción.

**Ej: solo el rol ADMIN puede eliminar un producto del catálogo.**



TRANSFORMACIÓN

# Habilitando Spring Security en el proyecto

- Se agrega una sola dependencia al pom.xml: spring-boot-starter-security.
- Spring Boot aplica autoconfiguración: automáticamente protege TODOS los endpoints de la aplicación.
- Al iniciar el proyecto, la aplicación ya exige autenticación (usuario "user" y una contraseña generada en consola).

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-security
  </artifactId>
</dependency>
```

**Importante**

Con solo agregar la dependencia, TODA la API queda bloqueada tras un login básico. Por eso el siguiente paso obligatorio es definir explícitamente usuarios, roles y qué rutas son públicas o privadas.

TRANSFORMACIÓN

# Paso 1: definir usuarios y roles

Para empezar a controlar el acceso, primero declaramos quiénes son los usuarios válidos del sistema y qué rol tiene cada uno.

```
@Configuration
public class SecurityConfig {
  @Bean
  public UserDetailsManager userDetailsManager() {
    UserDetails admin = User.builder()
        .username("admin")
        .password("{bcrypt}$2a$10$7EqJt...")
        .roles("ADMIN")
        .build();

    UserDetails vendedor = User.builder()
        .username("vendedor1")
        .password("{bcrypt}$2a$10$8FpKl...")
        .roles("VENDEDOR")
        .build();

    return new InMemoryUserDetailsManager(
        admin, vendedor);
  }
}
```

- InMemoryUserDetailsManager: útil para pruebas y demos rápidas.
- Cada UserDetails agrupa username, password y roles().
- roles("ADMIN") internamente se guarda como ROLE_ADMIN.
- En un proyecto real, esta información vendrá de una base de datos (lo veremos más adelante).



TRANSFORMACIÓN

# Codificación de contraseñas: nunca en texto plano

### Modo {noop} (solo para pruebas)

```
.password("{noop}clave123")
```

Guarda la contraseña en texto plano. Nunca debe usarse en producción: cualquiera con acceso a la configuración vería las claves reales.

### Modo {bcrypt} (buena práctica)

```
@Bean
PasswordEncoder passwordEncoder() {
  return new BCryptPasswordEncoder();
}
```

BCrypt aplica un hash de una sola vía + una "sal" aleatoria. La contraseña nunca se desencripta: al iniciar sesión, se compara el hash de lo ingresado contra el hash guardado.

TRANSFORMACIÓN

# Paso 2: configurar el SecurityFilterChain

Este bean centraliza TODAS las reglas de seguridad: qué rutas son públicas, cuáles requieren login y cómo se maneja el formulario de acceso.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http)
    throws Exception {

  http.authorizeHttpRequests(auth -> auth
      .requestMatchers("/api/auth/**").permitAll()
      .requestMatchers("/api/productos/**").authenticated()
      .anyRequest().authenticated()
    )
    .httpBasic(Customizer.withDefaults())
    .csrf(csrf -> csrf.disable());

  return http.build();
}
```

- authorizeHttpRequests(): define las reglas de acceso.
- permitAll(): ruta abierta, sin login.
- authenticated(): exige haber iniciado sesión.
- httpBasic(): habilita autenticación básica (ideal para APIs REST consumidas por Postman/Angular).



TRANSFORMACIÓN

# Roles y Authorities: el mismo concepto, dos caras

- Un "role" (rol) es simplemente un tipo especial de "authority" (permiso).
- Internamente, Spring Security SIEMPRE antepone el prefijo ROLE_ a los roles.
- roles("ADMIN") en el código equivale a la autoridad ROLE_ADMIN.
- Los nombres de rol pueden ser cualquiera: ADMIN, VENDEDOR, CLIENTE, SUPERVISOR... deben mantenerse consistentes en todo el proyecto.

**Ejemplo — TechStore Perú SAC**

| | |
|---|---|
| admin1 | ROLE_ADMIN |
| vendedor1 | ROLE_VENDEDOR |
| cliente1 | ROLE_CLIENTE |

TRANSFORMACIÓN

# Paso 3: proteger rutas según el rol

requestMatchers() define el patrón de ruta; hasRole()/hasAnyRole() define quién puede acceder.

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/productos/**")
        .hasAnyRole("ADMIN", "VENDEDOR")
    .requestMatchers(HttpMethod.DELETE, "/api/productos/**")
        .hasRole("ADMIN")
    .requestMatchers("/api/pedidos/**")
        .hasAnyRole("ADMIN", "VENDEDOR", "CLIENTE")
    .requestMatchers("/api/auth/**").permitAll()
    .anyRequest().authenticated()
);
```

- El orden de las reglas importa: Spring evalúa de arriba hacia abajo y aplica la primera coincidencia.
- "/**" incluye la ruta y todas sus subrutas.
- anyRequest().authenticated() debe ir siempre al final, como regla de respaldo.

hasRole("ADMIN") → exige exactamente ese rol. hasAnyRole("A","B") → acepta cualquiera de la lista.



TRANSFORMACIÓN

# Seguridad a nivel de método: @PreAuthorize

- Además de proteger rutas en el SecurityFilterChain, podemos proteger un método específico del Controller o del Service.
- Se habilita con @EnableMethodSecurity en la clase de configuración.

Dos capas, un mismo objetivo: SecurityFilterChain protege por RUTA, @PreAuthorize protege por MÉTODO.

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig { ... }

@RestController
@RequestMapping("/api/productos")
public class ProductoController {

  @PreAuthorize("hasRole('ADMIN')")
  @DeleteMapping("/{id}")
  public void eliminar(@PathVariable Long id) {...}

  @PreAuthorize("hasAnyRole('ADMIN','VENDEDOR')")
  @PostMapping
  public Producto crear(@RequestBody Producto p) {...}
}
```

TRANSFORMACIÓN

# De memoria a base de datos: UserDetailsService

En un proyecto real, los usuarios y roles viven en la base de datos, no hardcodeados en Java.

```
@Service
public class UsuarioDetailsService implements UserDetailsService {

  @Autowired
  private UsuarioRepository usuarioRepository;

  @Override
  public UserDetails loadUserByUsername(String username) {
    Usuario u = usuarioRepository.findByUsername(username)
        .orElseThrow(() ->
            new UsernameNotFoundException("Usuario no encontrado"));

    return User.builder()
        .username(u.getUsername())
        .password(u.getPassword())      // ya guardada con BCrypt
        .roles(u.getRol())               // ej: "ADMIN"
        .build();
  }
}
```

- Se implementa la interfaz UserDetailsService.
- Spring Security la invoca automáticamente en cada intento de login.
- La entidad Usuario y su UsuarioRepository se crean igual que Producto en la Sesión 6 (JPA/Hibernate).
- El campo rol puede mapearse a una tabla propia si un usuario necesita varios roles.



TRANSFORMACIÓN

# Manejo de acceso denegado

### 401 Unauthorized

El usuario NO se autenticó (no inició sesión o sus credenciales son inválidas).

### 403 Forbidden

El usuario SÍ se autenticó, pero su rol no tiene el permiso necesario para ese recurso.

**Personalizar la respuesta de acceso denegado:**

```
.exceptionHandling(ex -> ex
    .accessDeniedPage("/api/errores/sin-permiso")
);
```

*En una API REST, en lugar de una página HTML, normalmente se devuelve un JSON con el código de error y un mensaje claro para el cliente (Angular, Postman, etc.).*

TRANSFORMACIÓN

# Ejemplo aplicado 1: proteger el panel de administración

Caso: TechStore Perú SAC necesita que solo el rol ADMIN gestione el catálogo de productos.

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.GET, "/api/productos/**")
        .permitAll()
    .requestMatchers(HttpMethod.POST, "/api/productos")
        .hasRole("ADMIN")
    .requestMatchers(HttpMethod.PUT, "/api/productos/**")
        .hasRole("ADMIN")
    .requestMatchers(HttpMethod.DELETE, "/api/productos/**")
        .hasRole("ADMIN")
);
```



**Regla de oro**

Distinguir por método HTTP (GET/POST/PUT/DELETE) permite reglas muy precisas: leer es público, escribir es privado.

Resultado: cualquier visitante puede VER el catálogo (GET), pero solo un ADMIN autenticado puede crear, editar o eliminar productos.



TRANSFORMACIÓN

# Ejemplo aplicado 2: tres roles, tres capacidades

Caso: TechStore define CLIENTE, VENDEDOR y ADMIN sobre el módulo de pedidos.



**CLIENTE**

Crea sus propios pedidos y consulta solo su historial de compras.



**VENDEDOR**

Consulta todos los pedidos y actualiza su estado (enviado, entregado).



**ADMIN**

Tiene acceso total: además, puede anular pedidos y ver reportes.

```
.requestMatchers(HttpMethod.POST, "/api/pedidos")
    .hasAnyRole("CLIENTE","VENDEDOR","ADMIN")
.requestMatchers(HttpMethod.PUT, "/api/pedidos/**/estado")
    .hasAnyRole("VENDEDOR","ADMIN")
.requestMatchers(HttpMethod.DELETE, "/api/pedidos/**")
    .hasRole("ADMIN")
```

TRANSFORMACIÓN

# Ejemplo aplicado 3: usuarios reales desde la base de datos

Caso: conectar la tabla usuarios de TechStore con Spring Security usando JPA.

```
@Entity
@Table(name = "usuarios")
public class Usuario {
  @Id @GeneratedValue
  private Long id;
  private String username;
  private String password;   // guardada con BCrypt
  private String rol;        // "ADMIN", "VENDEDOR", "CLIENTE"
}

public interface UsuarioRepository
    extends JpaRepository<Usuario, Long> {
  Optional<Usuario> findByUsername(String username);
}
```

**Conexión con lo aprendido**

Esta entidad Usuario y su Repository se crean exactamente igual que Producto en la Sesión 6. Spring Security solo necesita que UsuarioDetailsService (slide anterior) las conecte al mecanismo de login.



PRÁCTICA

# Hora de practicar

Resolverán 3 ejercicios progresivos sobre el mismo caso: TechStore Perú SAC. Usarán lo aprendido hoy y en sesiones anteriores.

**Inicial**

Configurar un usuario en memoria y proteger una ruta simple.

**Intermedio**

Definir múltiples roles y reglas por método HTTP.

**Avanzado**

Conectar usuarios desde base de datos con BCrypt y @PreAuthorize.

PRÁCTICA

NIVEL INICIAL

# Ejercicio 1: la primera puerta protegida

TechStore quiere que el endpoint GET /api/reportes/ventas solo pueda ser consultado por un usuario autenticado con el rol ADMIN. Aún no existe ningún usuario configurado.

**Tu tarea:**

- Declara en memoria un usuario "gerente1" con contraseña cifrada con BCrypt y rol ADMIN.
- Configura el SecurityFilterChain para que solo ese rol acceda a /api/reportes/**.
- El resto de rutas de la API deben permanecer accesibles solo para usuarios autenticados (sin exigir rol específico).



PRÁCTICA

NIVEL INTERMEDIO

# Ejercicio 2: tres roles, un módulo de inventario

El módulo de inventario de TechStore expone: GET /api/inventario/** (consultar stock), PUT /api/inventario/{id}/ajuste (ajustar stock manualmente) y POST /api/inventario/alerta (crear alerta de reabastecimiento). Existen los roles CLIENTE, ALMACENERO y ADMIN.

**Tu tarea:**

- Los CLIENTE no deben tener acceso a ningún endpoint del inventario.
- ALMACENERO y ADMIN pueden consultar y ajustar el stock (GET y PUT).
- Solo ADMIN puede crear alertas de reabastecimiento (POST).
- Escribe el bloque authorizeHttpRequests() completo con estas reglas, en el orden correcto.

PRÁCTICA

NIVEL AVANZADO

# Ejercicio 3: usuarios reales desde la base de datos

TechStore ya tiene una tabla usuarios (id, username, password, rol) poblada con datos reales. El equipo necesita reemplazar el InMemoryUserDetailsManager por una integración real con JPA, y además proteger el endpoint de eliminación de productos solo a nivel de método (no solo por ruta).

**Tu tarea:**

- Implementa UsuarioDetailsService a partir de UsuarioRepository (usa findByUsername).
- Configura el PasswordEncoder con BCryptPasswordEncoder.
- Habilita @EnableMethodSecurity y protege eliminarProducto() con @PreAuthorize("hasRole('ADMIN')").



PRÁCTICA

# Solución: Ejercicios 1 y 2

**Ejercicio 1**

```
UserDetails gerente = User.builder()
  .username("gerente1")
  .password(passwordEncoder.encode("clave123"))
  .roles("ADMIN")
  .build();
// ...
.requestMatchers("/api/reportes/**").hasRole("ADMIN")
.anyRequest().authenticated()
```

*El resto de rutas queda cubierto por anyRequest().authenticated(), que exige login sin exigir un rol específico.*

**Ejercicio 2**

```
.requestMatchers(GET,"/api/inventario/**")
    .hasAnyRole("ALMACENERO","ADMIN")
.requestMatchers(PUT,"/api/inventario/**")
    .hasAnyRole("ALMACENERO","ADMIN")
.requestMatchers(POST,"/api/inventario/alerta")
    .hasRole("ADMIN")
.anyRequest().authenticated()
```

*CLIENTE nunca aparece en la lista, por lo que queda bloqueado por defecto ante cualquier ruta de inventario.*

PRÁCTICA

# Solución: Ejercicio 3

```
@Bean
public UserDetailsService userDetailsService(
    UsuarioRepository usuarioRepository) {
  return username -> {
    Usuario u = usuarioRepository.findByUsername(username)
        .orElseThrow(() ->
            new UsernameNotFoundException("No existe"));
    return User.builder()
        .username(u.getUsername())
        .password(u.getPassword())
        .roles(u.getRol())
        .build();
  };
}

@Bean
public PasswordEncoder passwordEncoder() {
  return new BCryptPasswordEncoder();
}
```

```
@EnableMethodSecurity
public class SecurityConfig { ... }

@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/{id}")
public void eliminarProducto(
    @PathVariable Long id) {
  productoService.eliminar(id);
}
```

*Clave: el explorador/cliente del DELETE ahora es rechazado en TIEMPO DE EJECUCIÓN del método, aunque hubiera pasado la validación de ruta.*



CIERRE

# Dinámica: "La cadena de seguridad"

En cadena, cada estudiante dice UNA idea clave de la sesión sin repetir lo que dijeron sus compañeros anteriores: un concepto, una anotación, un método o un riesgo evitado. El docente va anotando las ideas en la pizarra.

**Ideas que deberían aparecer en la cadena:**

Autenticación · Autorización · SecurityFilterChain · requestMatchers · hasRole / hasAnyRole · BCrypt · UserDetailsService · @PreAuthorize · 401 vs 403

CIERRE

# Quiz de salida: 3 preguntas

**1. ¿Qué hace exactamente hasRole("ADMIN") en una regla de requestMatchers()?**

a) Verifica que el usuario exista en la BD

b) Exige que el usuario autenticado tenga el rol ROLE_ADMIN

c) Cifra la contraseña del usuario

d) Permite el acceso a cualquier usuario autenticado

**2. Un usuario ingresa credenciales inválidas. ¿Qué código de estado HTTP es el esperado?**

a) 200 OK

b) 403 Forbidden

c) 401 Unauthorized

d) 500 Internal Server Error

**3. ¿Cuál es la principal ventaja de BCryptPasswordEncoder frente a {noop}?**

a) Es más rápido de escribir

b) Aplica hash de una vía con sal, evitando exponer la contraseña real

c) Permite recuperar la contraseña original

d) No requiere configuración adicional



CIERRE

# Casos prácticos retadores

**Caso A**

Un desarrollador junior de TechStore configuró: requestMatchers("/api/**").hasRole("ADMIN") ANTES que requestMatchers("/api/auth/login").permitAll(). Al probar el login desde Postman, siempre recibe 403. ¿Qué está mal y cómo lo corriges?

**Caso B**

Susana (rol VENDEDOR) reporta que puede eliminar productos desde Postman enviando DELETE /api/productos/12, cuando esa acción debería ser exclusiva de ADMIN. Revisando el código, solo existe @PreAuthorize en el método, sin regla en el SecurityFilterChain. ¿Por qué podría estar fallando la protección y qué revisarías primero?

# ¡Excelente trabajo hoy!

Hoy dieron un paso decisivo: sus APIs dejaron de ser proyectos abiertos y se convirtieron en aplicaciones profesionales que protegen datos reales. Practiquen configurando roles en su propio proyecto integrador antes de la próxima sesión, donde veremos autenticación con JWT.

**Gracias por su participación activa en clase.**

*Los espero en todas las sesiones y en el foro del curso ante cualquier consulta. ¡Sigamos construyendo!*

**Próxima sesión: JWT** | **Unidad 2 · Semana 9**